

ABM bug review, 17 April 2026 (v2)

Pre-existing issues found during a full-repo audit and independently verified against the code and data. Two regressions introduced earlier today (clustering and stale scenario YAML) were already fixed and pushed. This document lists the remaining issues for team triage.

At a glance

	Severity	Affects	Decision needed
C3	Critical	network topology	Drop the AP to A fuzzy match?
M1	Major	agent routing	Accept approximation or improve?
M2	Major	diagnostics only	Quick win
M5	Major	scenario runner	Clean up demo vs YAML
m2	Minor	optimization edge case	Defensive code?
m3 to m7	Minor	polish and docs	Optional

Critical

C3. AP-X chargers falsely appearing on A-X corridor (and vice versa)

Where	<code>src/new-abm/data_generation/spanish_network.py</code> , lines 528 to 532 (<code>_normalise_road</code>) and 110 to 143 (corridor list).
What is happening	<code>_normalise_road</code> strips the P from AP-X, so the fuzzy-match step in <code>_cluster_chargers_on_road</code> treats AP-2 and A-2 as the same road. The raw charger data tags them distinctly (AP-2: 10 chargers, A-2: 409). The fuzzy match causes BOTH corridors to receive ALL 419 chargers.
Why that is wrong	AP-2 and A-2 are separate parallel highways. AP-2 is the autopista (originally a toll road, now free); A-2 is a different physical <i>autovía</i> . A driver on AP-2 cannot use A-2 service areas without exiting and re-entering. Same applies to AP-4 / A-4, AP-1 / A-1, AP-68 / A-68, AP-9 / A-9, AP-46 / A-46.
Measured	185 physical charger locations end up represented as multiple station nodes. Connector capacity at those locations is effectively doubled: the physical chargers get two independent SimPy queues instead of one.
Important	The <i>demand</i> side is fine. AP-2 and A-2 have distinct per-segment demand rows in the CSV (3 vs 20 segments), so the 12 MAD to BCN style 'duplicate' OD pairs are actually correct parallel-road demand, not duplication. The bug is only in charger assignment.
History	Present since the first <code>build_spain_real_network</code> commit (15 April). Not introduced by today's work.
Options	A. Remove the AP- to A- regex. Keep the NSEW suffix strip (that handles A-7 / A-7S, which really are the same road). One-line fix. B. Leave as is, accept the 2x station capacity on parallel corridors as a modelling shorthand. C. More invasive: ensure every AP / A pair has its own entry in <code>_ROAD_CORRIDORS</code> with the correct physical chargers. Today some AP- corridors do not appear at all (no AP-4 entry), so removing fuzzy-match would orphan their chargers; the corridor list would need to be audited.

Major

M1. Expected wait time over-estimates by 1.4 to 5x

Where	src/new-abm/models/station.py, lines 89 to 101 (expected_wait_time_min).
Issue	Formula is $(\text{queue} + \text{current_users}) / \text{num_connectors} * \text{avg_session}$. When queue is 0 but all c connectors are busy, it returns a full session (22 min). Correct M/D/c expectation for a new arrival in that state is the residual service time of the first connector to free, roughly $\text{avg_session} / (c + 1)$.
Measured bias	queue=0, 4/4 connectors busy: formula 22 min, correct ~4.4 min (5x high). queue=4, 4/4 busy: formula 44 min, correct ~26 min (1.67x high). queue=8, 4/4 busy: formula 66 min, correct ~48 min (1.36x high). Over-estimation is worst when queue is small, which is exactly when the decision matters most.
Impact	Agents over-estimate waits at busy-but-moving stations and skip to the next stop, which may itself be more congested. Plausibly contributes to the fat-tail pattern observed today (90% of charge events have 0 min wait but a tail exceeds hundreds of minutes).
Options	A. Change to $\text{queue} / c * \text{avg_session} + \text{avg_session} / (c + 1)$. One line. B. Track residual service time per connector and return the minimum. More accurate, more code. C. Keep the high bias deliberately as a 'driver pessimism' behavioural layer. Defensible but should be documented as such.

M2. Stranding reason never populated in output CSV

Where	src/new-abm/simulation/engine.py, _record_strand() at lines 378 to 399, and the results collector schema.
Issue	agent.failure_reason is a dataclass field. It is set to "no_path_found" only in one place (the failed-route path at line 102). The SOC-depletion strand path (_record_strand) never writes it, and the output CSV (baseline_trip_records.csv) has no failure_reason column at all.
Impact	Debugging stranded agents requires grepping warning logs. The ABM-scaling debug procedure in memory/abm_scaling_task.md explicitly asks for breakdown by failure_reason, which is not currently possible from the CSV alone.
Options	A. Add agent.failure_reason to the strand paths with values like soc_depleted, no_reachable_station, sim_window_timeout. Also add the column to the collector schema. Self-contained, no downside.

M5. Two of three demo scenarios are silent no-ops on the real network

Where	src/new-abm/scenarios/runner.py (make_demo_scenarios, lines 178 to 204) and src/new-abm/config/scenarios/*.yaml.
Issue	price_reduction and capacity_increase both reference synthetic-network station IDs like STA_MAD_N, STA_ZAR_E. These exist only in build_spain_demo_network(), not in the real pipeline, so the scenarios match nothing. high_home_charging is fine because it changes a fleet-wide parameter, not station IDs.
Supporting evidence	In the 627k scenario comparison run earlier today, price_reduction and capacity_increase returned numbers identical to baseline, consistent with silent no-ops.

Related	config/scenarios/price_reduction.yaml and capacity_increase.yaml also reference the same stale synthetic IDs and are never loaded by run_scenarios.py anyway (only summer_peak and expand_hubs are).
Options	A. Regenerate the two scenarios with real station IDs (like we did for expand_hubs.yaml). Load them from YAML, delete the hardcoded versions. B. Remove these scenarios entirely if nobody plans to use them. C. Keep them but explicitly scope to the synthetic demo mode only and document that.

Minor

m1. Stale substation count (2,137 vs 2,147)

Where	A few historical entries in memory/decisions_log.md.
Impact	Docs only. Pipeline outputs correctly use 2,147 everywhere. Cosmetic.

m2. tent_tier fallback edge case

Where	src/optimization.py, around line 91.
Issue	If a row had is_tent=True together with tent_tier in ('none', ''), spacing would fall through to 120 km instead of 60 km (TEN-T Core).
Current state	Verified against demand_per_segment.csv: no such row exists today. Latent bug, not currently active.

m3. Misleading 'make this a generator' comments

Where	src/new-abm/simulation/engine.py, lines 356 and 375 (_handle_emergency_charge).
Issue	Both places have yield # make this a generator after an unreachable return. The function is already a generator thanks to real yields earlier in the body, so these dead yields serve no purpose. Code works, but the comments mislead future editors.
Impact	Cosmetic. Delete the dead yield lines and fix the comments when convenient.

m4. Stale 5.7% / 5.71% references in docs

Where	src/abm_demand.py:88 docstring; references/assumptions.md lines 337, 339, 429; references/github_roadmap.md:107.
Issue	Docstring and docs cite EV penetration as 5.71% (based on the old 2 M EV base). Actual EV_PENETRATION_RATE is $2,498,159 / 35,000,000 = 7.14\%$. Code uses the correct value dynamically; only the narrative explanations are stale.
Impact	Docs only. Can be updated any time.

m5. apply_scenario silently skips unknown station IDs

Where	src/new-abm/scenarios/base_scenario.py, line 118 (.get(sid, 0) pattern).
Impact	A typo in a scenario YAML fails silently (no warning, no error). This is why M5 and today's stale expand_hubs.yaml IDs went unnoticed. Adding a warning when scenario IDs do not match any real station would surface future regressions immediately.

m6. No unit test for `no_dest_charger_arrival_soc_fraction`

Where	Introduced today in <code>behavior/routing.py</code> and <code>behavior/en_route.py</code> ; no matching test in <code>src/new-abm/tests/</code> .
Impact	A future refactor could silently orphan the config key. Low-risk given we validated the behaviour manually today, but worth adding coverage.

m7. Silent 100 km fallback in `_project_onto_polyline`

Where	<code>src/new-abm/data_generation/spanish_network.py</code> lines 813 and 838.
Impact	If a city node is missing from the network, the code uses 100 km as the default segment distance without logging a warning. Never fires today (all referenced cities exist), but defensible to at least log a warning.

Verified NON-issues (do not spend time on these)

Bidirectional OD duplication (previously 'M4'). Withdrawn. AP-2 and A-2 are genuinely separate parallel highways in Spain carrying distinct traffic. The demand CSV has different trip counts for each. Having separate OD pairs per corridor is the right model, not a bug.

'return without yield at engine.py:355 breaks the generator'. The function has real `yield` statements further down the body, so Python treats it as a generator regardless. Related code-tidiness concern is tracked as m3.

'SOC calculation order risky at engine.py:285'. Order `energy_added = target - current` then `current = target` is mathematically correct.

'Bidirectional pairs double minority-direction demand'. Demand CSV is per-segment aggregate (not directional), so splitting it evenly across both directions is the intended model.

Suggested order of attack

1. M2. Stranding reason. Self-contained, pure diagnostic, unlocks easier ABM debugging. No modelling implications.

2. M5 plus m5. Scenario cleanup. Regenerate `price_reduction.yaml` and `capacity_increase.yaml` against real station IDs, load them from YAML in `run_scenarios.py`, add a warning in `apply_scenario` when a station ID does not match. Removes a class of silent failures.

3. M1. Wait-time formula. Plausibly improves the fat-tail wait pattern. Needs a little discussion on which approximation to use.

4. C3. AP / A fuzzy match. Biggest call. Dropping the AP- to A- regex is mechanically one line, but materially changes charger distribution on parallel corridors (AP-2 goes from 419 chargers to 10, A-2 stays at 409). All scenario results produced so far would need a re-run and re-interpretation. Worth a 10-minute call before touching.

Already fixed and pushed earlier today: clustering regression (*16-floor added back*) and `expand_hubs.yaml` (*regenerated with current station IDs*).